

Article

Can Large-Language Models Replace Humans in Agile Effort Estimation? Lessons from a Controlled Experiment

Luka Pavlič ^{*}, Vasilka Saklamaeva  and Tina Beranič 

Faculty of Electrical Engineering and Computer Science, University of Maribor, Koroška Cesta 46, 2000 Maribor, Slovenia; vasilka.saklamaeva@um.si (V.S.); tina.beranic@um.si (T.B.)

* Correspondence: luka.pavlic@um.si

Abstract: Effort estimation is critical in software engineering to assess the resources needed for development tasks and to enable realistic commitments in agile iterations. This study investigates whether generative AI tools, which are transforming various aspects of software development, can improve effort estimation efficiency. A controlled experiment was conducted in which development teams upgraded an existing information system, with the experimental group using the generative-AI-based tool GitLab Duo for estimation and the control group using conventional methods (e.g., planning poker or analogy-based planning). Results show that while generative-AI-based estimation tools achieved only 16% accuracy—currently insufficient for industry standards—they offered valuable support for task breakdown and iteration planning. Participants noted that a combination of conventional methods and AI-based tools could offer enhanced accuracy and efficiency in future planning.

Keywords: software engineering; agile development; iteration planning; effort estimation; generative AI; tool accuracy



Citation: Pavlič, L.; Saklamaeva, V.; Beranič, T. Can Large-Language Models Replace Humans in Agile Effort Estimation? Lessons from a Controlled Experiment. *Appl. Sci.* **2024**, *14*, 12006. <https://doi.org/10.3390/app142412006>

Academic Editors: Linda Vickovic and Maja Braović

Received: 15 September 2024

Revised: 1 December 2024

Accepted: 18 December 2024

Published: 22 December 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

According to the Cambridge Dictionary [1], estimation is an opinion on something, resulting in an estimate (or estimation, noun), which is a guess or calculation about the cost, size, value, etc., of something. Estimates performed in project management and software engineering are far from guesses, since they dictate the project schedules, finances and expected results.

Modern project management sees estimation as a part of the initial project schedule management [2], where estimating the duration of particular project activities leads to assessing the complete project duration, cost and resources that are required to fulfil the project. Project estimation is a well-defined and carefully executed set of activities carried out with a selection of proven tools and techniques that include expert judgement, estimating with analogies, parametric estimation and others [2]. Software engineering is no exception. Estimation is connected to assessing effort, resulting in a finances, schedule and resources plan.

Traditionally, in software engineering, effort is estimated to plan development and maintenance activities. In conventional (i.e., sequential), long-term requirement-driven development projects, effort estimation is usually based on a given set of requirements (or increments). In this setting, effort estimation is targeting longer periods and is, therefore, crucial to the project level and can be a source of many risks connected to misestimation. The issues connected to (mis)estimation have been known for decades and have been exposed and explained in several texts, including Brooks' "The Mythical Man-Month" [3], where he exposed several examples of non-deterministic behaviour in development teams connected to team size, project duration and failed project deliverable acceptance. As he stated, "Estimating the time required for software development tasks accurately is

notoriously difficult”, which is the result of several interconnected aspects: the high level of complexity and uncertainty in software projects, the overly optimistic nature of software developers and their managers and frequent unexpected technical issues, combined with changing requirements and integration challenges. However, effort estimation in software projects is a well-developed area, full of proven techniques and tools that can be used for accurate estimates [4–10]. The authors include and explain expert judgement, analogy-based methods, use case points, linear regression, robust regression, neural nets, constructive agile estimation algorithms, COSMIC FP, silent grouping, Delphi methods, COCOMO methods, planning poker and others. The mentioned methods [4–10] are either time-consuming or fine-tuned for a particular project and/or development team. A novel approach to effort estimation that is based on generative AI is also emerging. Software development platforms introduced this possibility without fully investigating its potentials and drawbacks (please see Section 3).

As opposed to rigorous effort estimation in conventional development methods, modern methods of agile software development usually take a rather different approach. Some just-in-time-oriented methods (e.g., Kanban) just eliminated the need for a systematic effort estimation during the development of system maintenance. Other iterative and requirement-driven methods do not employ effort estimation to estimate project duration; they rather estimate fine-grained assets frequently (usually user stories). This is the case with the leading methods of agile development [11], e.g., Scrum [12], Scrumban [13], large scale Scrum (LeSS) [14] and development level of scaled agile framework [15]. The approach was also introduced decades ago in agile development methods such as extreme programming (XP) [16] and feature-driven development (FDD) [17]. The aim of such estimation is to give a fair and sincere commitment for an iteration scope, which is usually two weeks long. Decision-making personnel, managers, product owners, etc., in modern projects are usually not interested in assessments on requirements; they are rather interested in the current iteration commitment (i.e., the list of requirements that the team is guaranteeing to be delivered by the end of an iteration). This is why hour-based assessments today are not the only alternative. In addition, development teams use several techniques and tools to carry out the short-term estimations, including story-points, T-shirt size estimation, planning poker and others [11,18]. The usual expert-based methods for effort estimation (including planning poker) are two-fold. Firstly, they include splitting requirements (usually in the form of user stories) into tasks; secondly, requirement estimate is calculated indirectly after estimating and summarising all generated particular tasks [12]. The research presented in this paper employs the same approach.

The changed nature of effort estimation in agile development, having several positive aspects, even resulted from the Brooks advice (e.g., breaking down the task, frequent re-estimation, an iterative approach, systematically added buffers and others), also comes with drawbacks. One of the most obvious is an important slice of the development time devoted for estimations. When teams perform the estimations themselves, planning-Mondays are quite a labour consuming activity, which, in a two-week long iteration, can easily mean 5–10% of the developers’ time that could otherwise be used for a development. Several adjustments of by-the-book agile development methods are employed frequently to improve the development team’s effectiveness. One of them is also estimation that is not carried out by the development team. Expert judgements, combined with analogies, are a replacement that is employed frequently [8,9]. As we demonstrate in Section 3, several artificial intelligence (AI)-based techniques have already been used for decades for either relieving developers of effort estimation and/or improving estimation accuracy. However, the rapid AI development in recent years, including generative AI and large language models (LLM), resulted in automated tasks that were unimaginable for automation a decade ago. Automated effort estimation in combination with LLM tools is a tempting idea. However, using a novel and nonverified approach in such an important activity with confidence and trust can be risky. On the other hand, even the large group of LLM sceptics [19] have positioned an effort estimation as a task with low risks, when LLM-based

chatbot-automated (see Figure 1). Estimates are not hard to verify. However, it is important to have them be accurate, which classifies effort estimation to tasks that can be automated, according to the authors in [19]. At the end of the day, several software development platforms have already included the generative-AI- and LLM-based functionalities that enable task generation and their assessment [20].

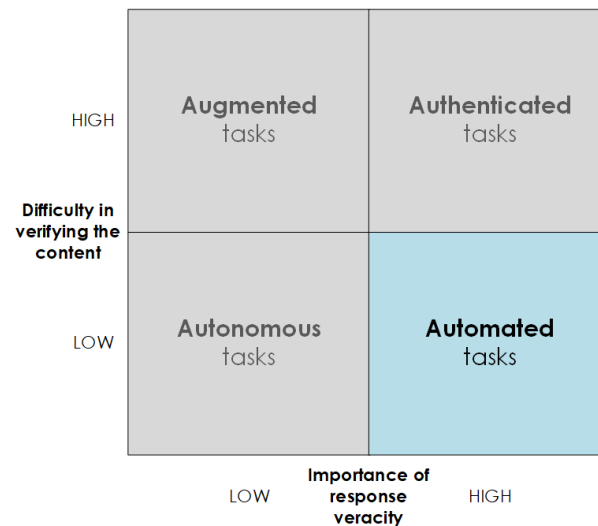


Figure 1. Positioning tasks that can be assisted by the LLM-based chatbots [19].

Based on the presented background, we decided to put a selected generative-AI- and LLM-based (AI-based) effort estimation assistant (see Section 3) on trial by employing a controlled experiment during an iteration-long development. The development teams were asked to split the requirements into tasks and estimate and implement them. We analysed the empirical data that we gathered during the experiment to compare conventional (using years-proven techniques and tools) and generative-AI-based (LLM-based chatbots) effort estimation while answering the following research questions:

- **RQ1:** Is there a difference between tasks created using conventional and generative-AI-supported tools?
 - **RQ1.1:** Is a set of tasks, created by generative-AI-supported tools, comparable to a set of tasks created using conventional methods?
 - **RQ1.2:** Are effort estimations created by generative-AI-supported tools comparable to effort estimations created using conventional methods?
- **RQ2:** How accurate is the effort estimation generated by a generative-AI-supported tool compared to conventional estimation methods?
- **RQ3:** Which aspects of effort estimation can be backed by generative-AI-supported tools?

The rest of the paper is organised as follows. In the next Section, we outline the field of employing AI-related techniques in effort estimation. Next, other authors' research related to ours is highlighted. We are employing generative AI as a tool for creating tasks, their estimation and possible verifications of such an approach. In Section 4, we describe our research method in detail, followed by stating the raw empirical data generated by the controlled experiment. Section 6 challenges the research questions with empirical data, and Section 7 outlines the main threats of validity. We conclude the paper with Section 8.

2. Current AI-Supported Effort Estimation

The history of AI-supported effort estimation is decades old. Since AI was always a field of moving targets, and AI itself is a term that redefines itself every few years, AI-supported tools and methods from the 1990s would not be classified as AI at all today (e.g., decision trees, analogy-recommended systems, case-based reasoning, etc.). There

are, however, some exceptions. One such example from 1996 [21] tried to employ artificial neural networks in estimating software development effort and the potential of case-based reasoning for development estimation. Using their simulated dataset, they were rather unsuccessful, with a prediction accuracy of less than 40%.

Later, AI-supported tools and methods employed primarily conventional machine learning models for effort estimation. An example study, reliable machine learning models for estimating effective software development efforts [22], compares the performance of eight different models for effort estimation, which were also not really successful in terms of estimation accuracy, which, in the best cases, reached up to 50% compared to conventional effort assessment methods.

The last five years systematic review [23] presents applications of AI techniques for software effort estimation. The authors compare and evaluate the performance of diverse machine learning models, including artificial neural network, support vector machine, linear regression, random forest, and other techniques. The proposed AI-based framework is claimed to hold the potential to enhance project planning and resource allocation.

In Ref. [24], research was performed, where the authors employed AI to predict an employee's workload from their reported work hours (having a decades long record on their reported working hours). So, the AI-based prediction in this research was on predicting actual effort, not the effort needed to perform a particular task.

Another review [18], which was based on 11 primary studies, was focused on data-driven effort estimation techniques of agile user stories. They identified several possible techniques and algorithms that are employed to enhance effort estimation, among them: Bayesian network, decision trees, support vector machines, long short-term memory and recurrent highway network, stacked autoencoders, variational autoencoders, denoising autoencoders, stacked denoising autoencoders, gradient boosted trees, k-nearest neighbours, artificial neural network, sridge regression, linear regression, naive Bayes, random forest, logistic model tree, and j48 decision tree, fuzzy based model and stepwise regression.

3. Related Work

The last five years systematic review study [23] did not capture any generative-AI-based approaches or tools. The aim of our research is to verify how generative-AI-based methods perform in real-world scenarios. The same applies to another review research [18], where generative-AI-based effort estimation was not identified.

However, we can find research in the field of employing generative AI for effort estimation. In one, the authors [25] describe the conceptual identification of the capabilities of LLM for cost estimation. They used a COSMIC function point estimation model as a prototype. An experiment that they conducted gives a comparison of a manual measurement with trained and untrained models. Their reported findings show that the cost estimation process can be accelerated by using large scale models. They showed how COSMIC function point mapping and measurement can be automated. Another niche study, employing generative AI in effort estimation [26], deals with intelligent interfaces and their user experience. They propose effort estimation based on generative AI in the form of a case study. The limitation of this study is that it is not generic to the field of effort estimation; it is focused towards enhancing the COSMIC function point model.

Arman et al. [27] introduced a conceptual solution designed to improve estimation accuracy and efficiency while minimising manual effort and cost. Their approach involved extracting entities and their relationships from the system, modelling them as semantic triples and developing conceptual micro-services. To enable a more precise functionality breakdown and more accurate effort estimation during microservice development, they incorporated prompt engineering with ChatGPT, enhancing requirements understanding, which, in turn, improved the accuracy of effort estimation. Generative AI in this case was not employed directly to perform the estimations.

A web-based AI chatbot named Alfred is a solution that the authors [28] introduced in their paper. It aids agile software release planning by employing two machine learning

models to estimate the duration of tasks and recommend optimal resource assignments for project managers. It is also able to categorise the task estimates based on their confidence level into three groups: low, medium, or high. Chatbot in this case is used as a user-friendly interface for underlying machine learning models. Barcaui and Monat [29] explored the bigger picture of project management in their research paper. They compared GPT-4 and a human project manager in developing project plans, analysing aspects like scope and schedule, as well as cost (effort) and resource estimation, which touched upon our research topic. The study found that AI and human plans have complementary strengths and weaknesses, and it highlighted the importance of human expertise in refining AI outputs. In this sense, the paper outlined similar takeaways as our study. However, our research was focused on task effort estimation carried out by developers using specialised tools.

The potential of generative AI in terms of planning iterations and estimating effort is also articulated clearly in the latest Gartner Research Reports [20]. Among others, they claim that by 2027, the number of platform engineering teams using AI to augment every phase of the SDLC will have increased from 5% to 40%. In addition, they reported in 2024 a list of the state-of-the-art automation tools available in certain points in the DevOps cycle. For planning the iteration based on generative AI, they identified the following tools and providers: Atlassian Intelligence, ClickUp, GitLab Duo and OpenText. Based on their recommendation and our own preliminary tool searching and testing, we decided to use GitLab Duo, which is integrated into GitLab Enterprise. Being listed as one of the leading tools, it gave us the confidence to generalise the findings of our research. Our research is also unique compared to others in terms that we have employed the generative AI tool for splitting user stories into tasks and, in addition, estimate tasks to obtain the complete effort estimation of an user story. This is also the case in several expert-based estimation techniques, e.g., planning poker, which gives our research direct possible validation with the influence on the industry.

4. The Research Method

To challenge the research questions that we defined in the Introduction, we designed a controlled experiment iteratively. We verified the first version using a small-setting pilot and refined the experiment. We followed a proven experiment scoping process, defined and implemented by several empirical research studies in software engineering [30,31]. The controlled experiment design (one factor experiment [30]) followed the objective of evaluating the performance of two competing treatments devoted to the same goal. In our case, it is the planning of an iteration in terms of splitting user stories into tasks and estimating effort. The experiment group was required to use a novel approach—an intelligent agent in a software development platform, while it was challenged by and compared with a control group, which was required to use conventional methods of task splitting and effort estimation of their choice (as they would usually—including planning poker, expert judgement and judgement with analogies). We invited higher-year students, the majority of them already working as developers in companies, to participate in the experiment. For participant profiles, please see Section 5. Figure 2 grabs the essence of our research method, focusing on the controlled experiment, activities, activity flow, independent variables and measured variables.

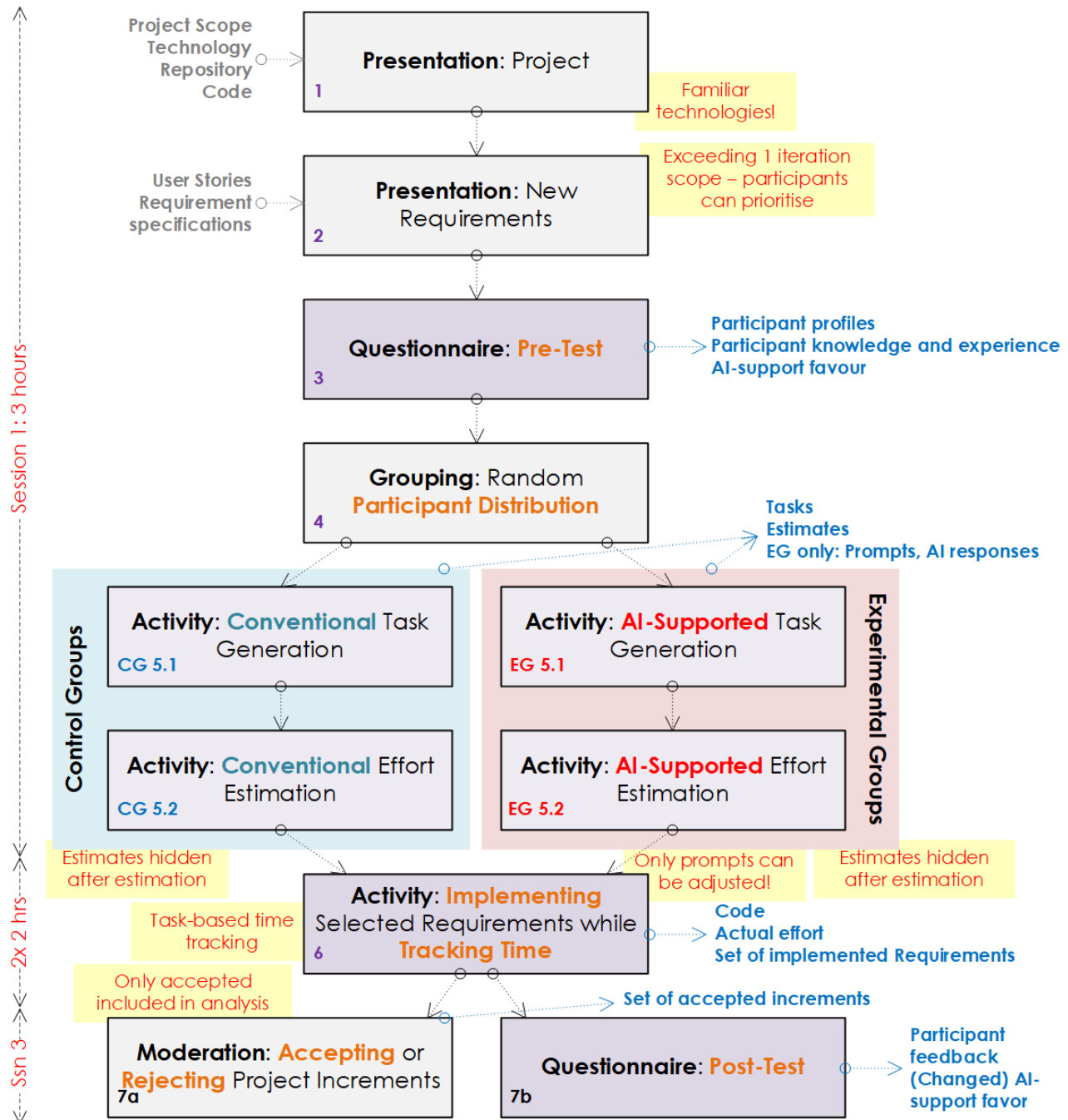


Figure 2. Research method outline with emphasis on controlled experiment.

The controlled experiment consisted, as explained in detail in the next subsection, of three sessions:

- **Session 1:** Requirement presentation, pre-test and participant distribution,
- **Session 2:** Requirement implementation with time tracking and
- **Session 3:** Accepting software increments, post-test.

The research method Figure is colour-coded. The responsibility for activities in the boxes with the gray background (1, 2, 4, 7a) were in the researcher. Purple-coloured boxes (3, 6, 7b) present the activities for participants, where boxes outlined in blue (5.1, 5.2) present activities for participants in the control group and red-outlined boxes (5.1, 5.2) are activities executed by experimental group participants.

The Experiment Setup

During our controlled experiment, the participants were asked to upgrade an existing information system with new functionalities. The system consisted of Jakarta EE (v10) backend, running on a Wildfly/JBoss application server (v.32), storing data in a relational database (configurable H2/MySQL/PSql via Java Persistence API) and a simple web-based user interface (Jakarta Server Faces). The architecture and technological stack of the system was standard and familiar to the experiment participants (please see Section 5 for details), which was checked additionally with the pre-test questionnaire (step 3, see Figure 2). Familiar technologies were chosen since we did not want results to be affected by technology-based drawbacks. Functionally, the system was part of a retailer internal application ecosystem with a simple aim: monitoring the temperature at which products are stored and alarming if a particular temperature measurement goes out of the allowed thresholds for the stored product. On the user interface, in this case, the measured temperature is marked as “not OK”. Some representative examples of the user interface can be seen in Figure 3.

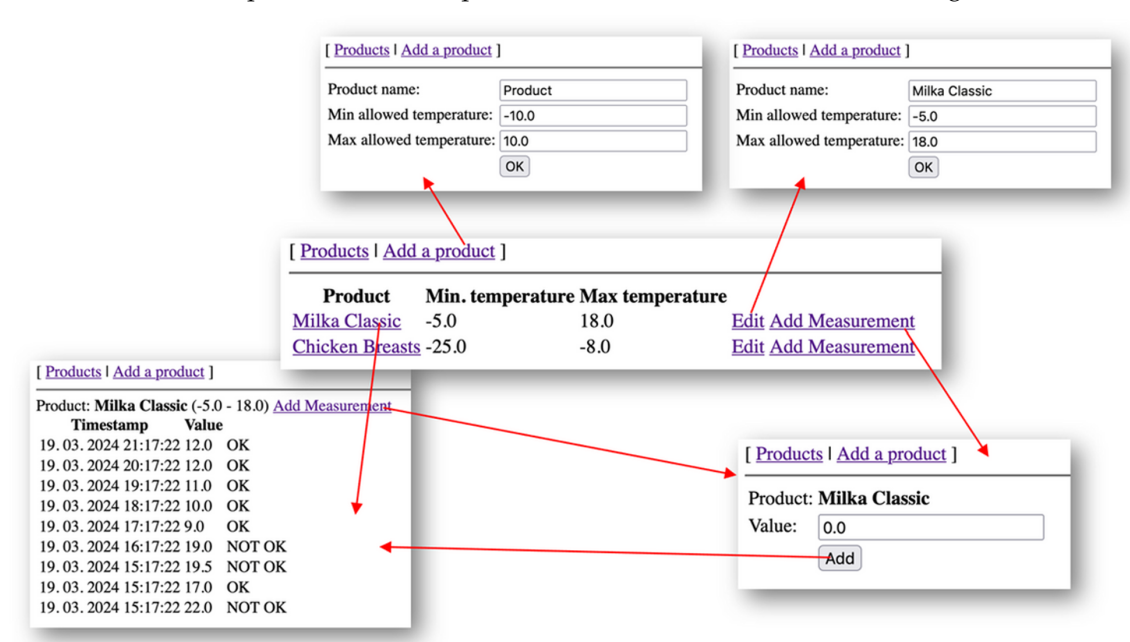


Figure 3. Sample flow of the system that participants were required to upgrade.

As a part of the experiment set-up, we prepared the participants with the initial push to a clean GIT repository (in our case GitLab), where we put all the source code and the documentation related to the system. We also handed them the documentation of the new requirements and the templates (spreadsheets) for the planning phase.

In experimentation session 1 (out of 3), which lasted approximately 3 h, we started with the presentation. During the presentation session (steps 1 and 2, see Figure 2), we presented the system in detail, performed a source code walk-through and demonstrated the development environment setup and initial run. We also presented the new requirements, which consisted of 8 user stories, accompanied by detailed specifications:

- **User Story #1: Email notification** in case of a measurement outside the tolerances. The participants were instructed that the notification should have included the following information: the product, its permitted temperature tolerances, the measurement taken and the exact time of the measurement. Additionally, they had to ensure that the email was sent from a dedicated email address.
- **User Story #2: Monthly reporting** of measurements outside of the allowed tolerances. The monthly report should have included all the measurements, no older than 30 days, that were outside the tolerances. They should have been organised by deviation from

the allowed temperature, in descending order, and should have also included the number of improper measurements for each product.

- **User Story #3: Colour coding** of the measurement history. The Table with the measurement history should have highlighted the inappropriate measurements in red and the appropriate measurements in green.
- **User Story #4: Support for the imperial and metric systems.** The participants were instructed to introduce settings in the menu that allowed switching between metric and imperial measurement systems. Depending on the choice, the user interface should also have displayed the label °C or °F throughout.
- **User Story #5: Display products on a graph.** We wanted to have the product graph in the application menu, which showed products based on their allowed temperature tolerances. This graph would later serve as a basis for highlighting products visually (and their temperature tolerances) in terms of their potential for shared storage.
- **User Story #6: Option to import data** from an offline temperature sensor. In case a specific measurement (date, time) has already been imported, the participants were instructed not to import the data again.
- **User Story #7: Temperature trend graph** for each product. Upon selecting a specific product, the participants should have enabled an alternative view of the data (in addition to tabular). They were not limited to a graph type. They were additionally instructed to ensure that the graph presented the tolerances and the actual temperature trends over time clearly.
- **User Story #8: Multilanguage support** (DE/EN/SI). Depending on the selection, the appropriate language of the user interface should have been displayed throughout the application.

The user stories were preliminarily estimated by experts, so we were confident that all user stories will not be implemented by any team. We deliberately proposed more user stories than would fit into a single development iteration so that the participants could make priorities and choose which functionalities to implement.

The next step in the first session was a pre-test questionnaire devoted to capturing the participants' knowledge, experience and possible positive or negative affinity towards generative-AI-based assistants (e.g., if a particular participant would be extremely and subjectively against AI-based assistants), since we wanted to have unencumbered individuals with a sincere intention to verify new methods and tools. The pre-test questionnaire included several questions, among them,

- What is your experience in software development with the Jakarta EE technologies?
- Which technologies do you master?
- How do you think the use of AI tools helps in software development?
- Have you used AI tools in software development before?
- Would you use AI tools to break down user stories into tasks and estimate them?

After the pre-test survey was finished successfully, **we split the participants randomly into two groups** and separated them physically in two different rooms. Both the experimental and control group participants were further combined in teams of three, which is a minimal setting to simulate a team in agile development. The project teams were asked to prepare tasks and estimate them. The same team that was performing the planning was also required to implement the user stories by following the tasks they created. Since the tasks were self-assessed, the team member pre-knowledge and experience were included implicitly in the effort estimation, which is also the case in the majority of the methods of agile development. The estimations were carried out in hours, since our setting was not spanning across many iterations, and, in addition, our research was interested in estimating accuracy, not in sincere iteration promises. The project teams were required to insert generated tasks into a GitLab project as issues and prepare a kanban board for the selected user stories. Although the teams were required to assess all 8 user stories, they were free to pick which and how many user stories they would like to implement in session 2. The

estimates that the teams produced were hidden from them since we did not want to have biased time tracking records.

In the case of the experimental group, a short introduction on GitLab Duo was performed before the effort estimation. During the experiment, a limited set of functionalities was used by the tool. The GitLab Duo Chat Boot was employed to provide the user story details and ask for splitting the user story into tasks with task estimates, thus providing a complete effort estimate for the user story. The project teams in that group were obliged to use this intelligent agent to produce both tasks and their estimations. They were not allowed to overrule the suggestions that the tool was giving, the only influence they had were their prompts that they fed to the chatbot. So, their task was also in fine-tuning prompts to obtain a satisfying response. In addition to the tasks and their effort estimates, we also recorded the prompts and responses that were used in the process of effort estimation in the experimental group. For prompts and response examples, please see Section 5.

Activity 6 (see Figure 2)—Implementing selected requirements while tracking time—took place in two separate dedicated timeslots, each approximately 2 h long. During that time, the development teams were asked to implement the tasks of the selected user stories. They were also required to track the time they needed to fulfil a single development task, which was carried out by GitLab—tracking time of moving issues on the kanban board (from “to do” to “in-progress” and from “in-progress” to “done”).

After the implementation finished, all the participants were asked to **fill the post-test questionnaire**. At the same time, we reviewed, checked and tested all the delivered user stories from all the development teams to accept the user stories that were claimed to be finished. In the data analysis (regarding the effort estimation accuracy), we only included the tasks and user stories that were accepted as completed correctly and according to requirements. Partially or faultily implemented increments were considered not to be implemented.

This is how our research method and controlled experiment design supported gathering the empirical data that were crucial to answer the research questions. The challenged variable was a method of task generation and their effort estimation, while the dependent variables included estimates (tasks, effort estimates, chatbot prompts, chatbot responses, set of delivered functionalities, actual development efforts) and questionnaire responses.

5. Results

The controlled experiment, in which 42 developers participated, was executed in April 2024. Their background included all the technologies that were used in the information system that they were asked to upgrade (all of them reported to master JSF—Jakarta Server Faces, EJB—Jakarta Enterprise Beans and JPA—Java Persistence API). Their development experience, both in study- or industry-level projects was, majorly, 1–3 years (91%). Three percent of the participants reported up to 5 years of experience, while six percent reported less than 1 year (see Figure 4). The participant involvement in the experiment was voluntary. The entry criterion was successful engagement in previous studying tasks, so only the most proven 42 students (out of 56) participated.

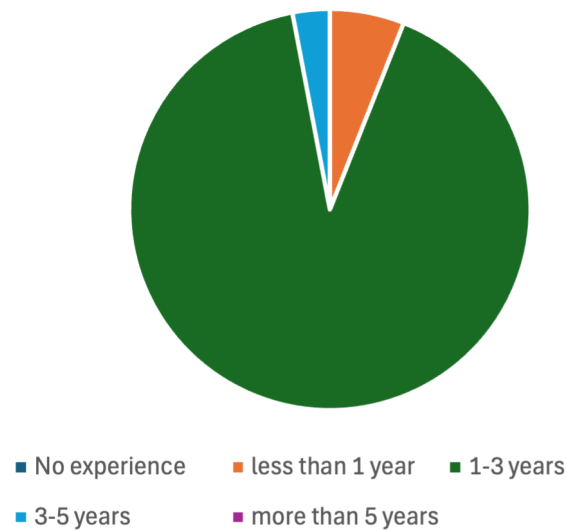


Figure 4. Participant experience.

The participants pre-test question answers are as follows. The majority of the participants had a positive attitude towards AI-based tools in software development, as demonstrated in Figure 5.

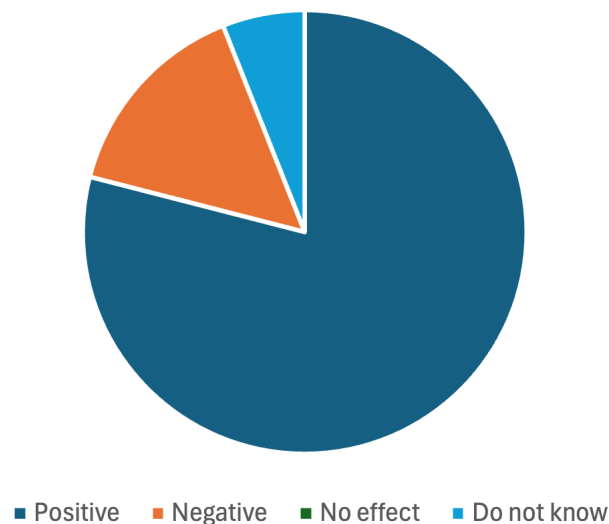


Figure 5. How do you think the use of AI tools helps in software development?

The great majority (82%) also reported that they had already used AI tools in software development. They listed the following tools: ChatGPT, GitHub Copilot, Claude and Superhuman. The participants were eager to use AI tools to break down user stories into tasks and estimate them—85%. Three percent did not like the idea, while twelve percent were undecided.

All 42 participants were divided randomly into the experimental and the control groups; inside the groups, they were also distributed randomly between development teams (7 teams in each group, 3 participants in each team). One team in the experimental group decided not to continue with the experiment, so we included and analysed the empirical data from 39 participants (6 development teams in the experimental group and 7 development teams in the control group).

5.1. Task Generation and Effort Estimation Phase

The participating development teams were given the eight user stories and were asked to prepare a sound set of tasks and their effort estimates. The development teams

generated 260 (experimental group) and 184 tasks (control group). As seen in Table 1, the development teams in the experimental group generated 43 tasks on average, while the control group development teams provided 29. An interesting number in Table 1 is also the average task and user story estimate. The user story estimate is calculated as a sum of all task estimates that belong to the user story (see Equation (1)).

$$\text{Estimate}(\text{US}_i) = \sum_{\text{US}_i, \text{tasks}} \text{Estimate}(\text{task}) \quad (1)$$

In the experimental group, the average task effort estimate is 2.5 h, while in the control group, the developers estimated tasks on average at less than half of that: 1 h. This led us to an average user story estimate that was, in the case of the experimental group, 13.5 h and, in the case of the control group, a quarter of that: 3.4 h.

Table 1. The number of generated tasks and estimation summary.

		Experimental Group	Control Group
Development teams		6	7
Estimated user stories per development team		8	8
Estimated user stories		48	56
Generated and estimated development tasks		260	184
Tasks per development team		43.33	26.29
Task estimate	average [h]	2.50	1.03
	st. dev	1.33	0.79
User story estimate	average [h]	13.55	3.37
	st. dev	6.02	1.76

An additional insight into effort estimates is given in Figures 6 and 7. Figure 6 shows the cumulative hours estimated by the experimental group teams (EG1–EG6) and the control group teams (CG1–CG7). We can see that the teams in the experimental group estimated significantly more effort (on average 108 h per team) compared to the control group teams (average 27 h).

Another perspective on cumulative effort estimates is presented in Figure 7. We can see that the user story average effort estimates (all the development teams) are considerably higher in all cases for the experimental group.

Let us demonstrate the difference in task content and individual effort estimates in the case of User Story #2 (Monthly reporting), with representative examples from the development teams in control in the experimental groups. The tasks generated by development team 6 (summarising in 6 h of estimate for User Story #2) in the control group were as follows:

- Monthly report button—0.5 h
- New Table—2 h
- Backend—review of all measurements within 30 days—0.5 h
- Backend—capturing measurements outside the tolerances—0.5 h
- Frontend—arrangement of the covered measurements descending—1 h
- Frontend—show inadequate measurements—1.5 h

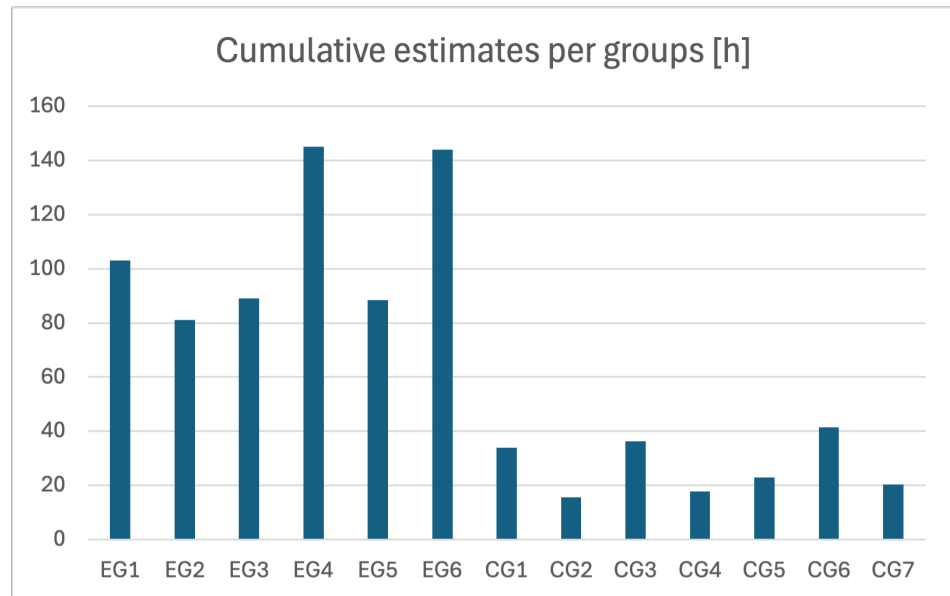


Figure 6. Cumulative effort estimates for all user stories per development teams in hours.

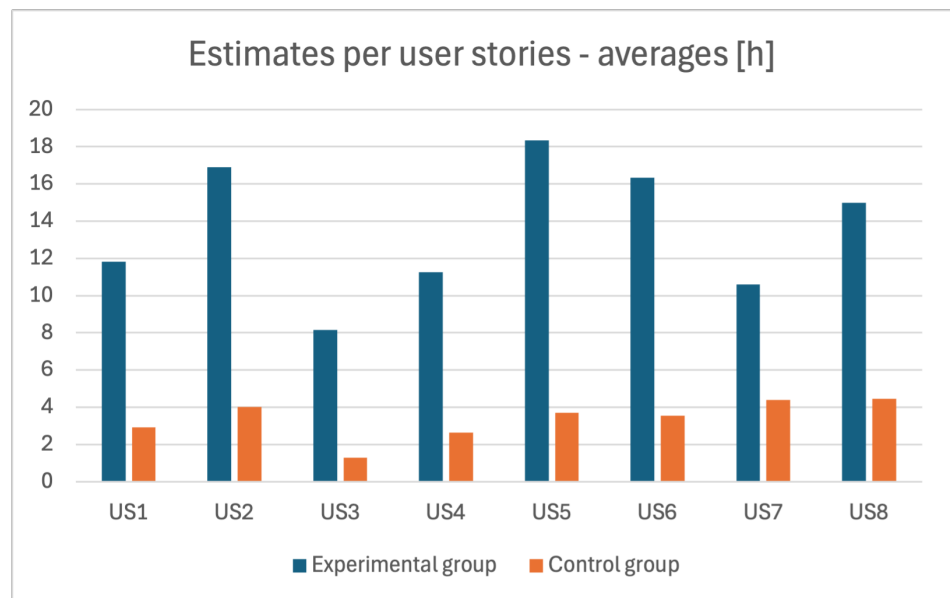


Figure 7. Average user story effort estimates per groups in hours.

While the development teams in the control group were given freedom on creating and estimating tasks (except for AI-tool support), the experimental group development teams had to use Gitlab Duo. They had to report not only the resulting tasks and estimates but also the prompts. Let us see some task sets for the same user story that demonstrate how a prompt plays an important role; we can also see that effort estimates are considerably higher.

Development team 1 used the following prompt to obtain the task set, which summarises a 14 h estimate for User Story #2, more than double compared to the control group development team:

Based on the existing code, write me tasks (and their time estimates in hours) to implement the following requirement: I need a new web page that will display a monthly report on how many times a temperature has been out of the tolerance for each product. The data must be displayed in descending order. The report must include several out-of-range

measurements for each product. When generating time estimates, be aware that I have some knowledge with the used technologies.

- Create a new page template and route—2 h
- Create a temperature model—3 h
- Aggregate the data and populate the model—4 h
- Update data aggregation job—2 h
- Display data on a new page—3 h

Another prompt example, used by development team 2, resulted in the tasks that summarise an 11 h estimate for User Story #2.

Based on your knowledge of this repository, generate different tasks (and their time estimate in hours) to implement the requirement: Monthly reporting on measurements outside the allowable tolerances. Add the option “monthly report” in the application menu. The monthly report should contain all measurements, up to 30 days old, that were outside the tolerance. They should be sorted by descending deviation from the allowed temperature. The report should also include the number of unacceptable measurements for each product.

- Add new “Monthly Report” menu option—1 h
- Extend the measurement model with a deviation field—2 h
- Implement monthly report logic—3 h
- Monthly report UI—3 h
- Testing and documentation—2 h

Development team 6 used almost the same prompt, which resulted in a 32h estimate for User Story #2:

- Add an interface for configuring the report parameters—2 h
- Implement a service to query the measurements and generate a report—8 h
- Implement sorting by temperature deviation—4 h
- Group report data by product—3 h
- Identify a location for the new menu option—1 h
- Unit tests for report generation logic—6 h
- Integration tests for end-to-end report generation—4 h
- Call monthly report service on the menu option selection—2 h
- Display the generated report—2 h

We gathered the tasks, effort estimates and prompts (in the case of the experimental group) for all the development teams and user stories. The prompts, cited before, are outlined in order to demonstrate the diversity that was produced not only by human generated tasks but also by the chatbot.

5.2. Implementation and Post-Test Phases

During the implementation, the development teams choose freely which user stories they would implement. After the implementation, we examined their deliverables and accepted or rejected the increments for a particular user story. We analysed only empirical data (the actual time spent on development tasks) generated for user stories that were both reported to be finished and accepted. The summary is shown in Table 2. The data show that despite the different effort estimates, they delivered a comparable number of user stories in the time given to perform the implementation. The actual time that was spent per task and user story was also similar, despite the different estimates.

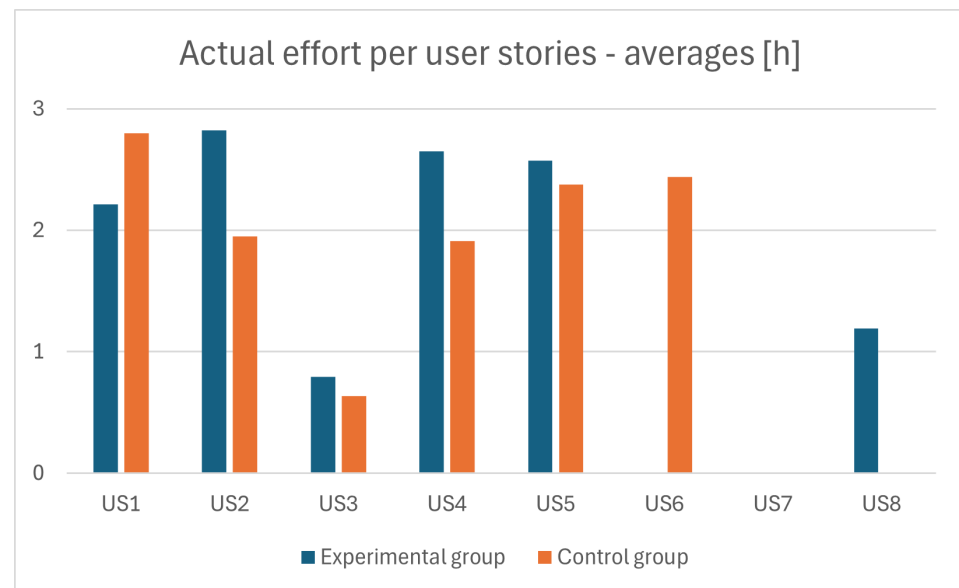
Table 2. Implemented user stories.

		Experimental Group	Control Group
Developed user stories		17	15
Developed user stories per development team		2.83	2.14
Actual effort per task	average [h]	0.67	0.61
	st. dev	0.91	0.59
Actual effort per user story	average [h]	2	1.92
	st. dev	1.7	1.26

Similarly to user story estimate, the actual effort of the user story is calculated as a sum of all the tasks' actual effort belonging to the user story (see Equation (2)). Please note that all tasks were not necessarily implemented, especially in the case of the experimental group. The unimplemented tasks' actual effort was reported to be 0 h.

$$\text{Actual effort}(US_i) = \sum_{US_i, \text{tasks}} \text{Actual effort}(\text{task}) \quad (2)$$

Another insight on the average actual effort per user story is shown in Figure 8. In the case of user stories 1 to 5, we can see effort from both the experimental and control groups. User stories 6, 7, and 8 show only the average development time in the control, experimental, or no group. The reason is simple: the actual implementation of User Story #6 was only carried out by development teams in the control group; User Story #8 was implemented only in the experimental group; and no-one decided to implement User Story #7. One can see that five User Stories' implementations, carried out by both groups, were problematic in terms of possible comparison, but, in fact, they were not. The empirical data actually boast of 32 user stories' implementations and 101 implemented tasks to perform the accuracy calculations and comparisons, seen in the next Section 6.

**Figure 8.** Average user story effort estimates per group in hours.

In parallel with judging the delivered system increments, the developers from the experimental group were asked to participate in a post-test questionnaire (see step 7b in Figure 2). We were interested in two answers:

- Do you think you would prefer to use AI tools, conventional methods or a combination of both in future projects? (Conventional methods—0%; AI tools—0%; A combination of both—100%)
- Do you think AI tools can provide more accurate task estimates compared to conventional methods? (Yes—10%; No—55%; Do not know—35%)

6. Discussion

Based on the research questions and empirical data exposed in the previous Section, we can exercise a grounded discussion. The participating developers demonstrated that they mastered the technologies used in the system, and, therefore, they did not suffer a technology-related handicap. They were also already familiar with AI-enabled tools while developing software—but not for effort estimation. However, they were also keen to try the tools for effort estimation. After the first session, the development teams provided task sets and their effort estimates, which enabled us to answer research question **RQ1: Is there a difference between tasks created using conventional and AI-supported tools?** The answer was positive—both in terms of content and in terms of estimates.

RQ1.1: Is a set of tasks created by generative-AI-supported tools comparable to a set of tasks created using conventional methods?

As demonstrated in Section 5, the most obvious difference between the groups is that the experimental group created 260 tasks (which averaged 5.4 tasks per user story), while the control group created only 184 (3.2 tasks per estimated user story). The task volume implies two conclusions:

- Human-generated tasks are not so fine-grained compared to the AI tool,
- The AI tool can generate more tasks in the same time—which is obviously related to effort estimation, which is time-consuming when carried out manually.

Performing a content-related analysis of tasks, we can see that in our experiment, human-generated tasks, in general, are a subset of AI-generated tasks. In general, the control group teams generated tasks that are required explicitly to fulfil the user story. On the other hand, in the experimental group, the teams reported tasks that were general to the development but were not required explicitly (e.g., unit tests, creating/updating documentation, refactoring existing code). Documentation and testing are tasks that can be found in the majority of experimental group task lists, and they add between 2 to 4 h to the estimate per user story. However, we can also see a lot of general tasks in the task lists that are not really related to user stories (e.g., Create a new page template and routing).

Based on the analysis of the empirical data, we would encourage development teams to start with their own list of tasks, since human-generated tasks would be more related to the context of the user story. Afterwards, an AI-generated task list can be of great help to see if the developers overlooked some tasks that were not so straightforward to spot.

RQ1.2: Are effort estimations, created by generative-AI-supported tools, comparable to effort estimations created using conventional methods?

In addition to the discussion on research question RQ 1.1, the experimental group not only generated considerably more tasks per user story but they were also estimated higher (see Table 1). The conventional effort estimates produced in our experiment were, on average, estimated per user story at 3.37 h, with an average task of 1 h. Contrarily, in the experimental group, we can see the average user story being estimated at 13.5 h, with tasks at an average estimate of 2.5 h. This is why we can see cumulative estimates in the experimental group teams that are considerably higher (108 h on average) compared to the control group (27 h)—see Figure 6.

Speaking strict statistics, when we compared the estimates of the same user stories by the experimental and control groups, the estimates were proven to have significant differences (two-tailed student *t*-test resulted in a *p*-value of 3.03×10^{-7} , *t*-value 6.676—see Table 3).

By analysing the tasks and their estimates closely, we can spot several possible causes for the really high difference between the estimates. Related to the discussion of research question RQ 1.1, an important number of estimates was added by tasks that the teams in

the control group did not envision (testing, documentation, refactoring). However, even with those tasks eliminated, we have an average user story estimate over 10 h. Another anomaly that we spotted in the experimental group compared to the control group were tasks that were not really context related, having also quite a high estimate (e.g., user story “Create a new page template and routing” was estimated at 2 h, while this task was not required in the project and was therefore not listed in the control group estimates). A third anomaly when comparing task estimates were the same tasks identified by both group teams but estimated higher by the experimental group teams. The reason for this might have been in (over)confident developers or in the AI-tool, which did not take into account the developers’ abilities and/or carried out the estimation in a worst-scenario manner. An example is a task from User Story #1 (Email Notification), which demands the developer prepare a custom, project-dedicated email (creating an email in some free online providers). In the control group teams, this task was estimated between a quarter and a half hour, while the AI-tool estimated this task at over 1 h, in the case of one team also 3 h, which was an obvious overestimate.

Let us discuss the estimate accuracy that we targeted with research question 2 (**RQ2: How accurate is the effort estimation generated by a generative-AI-supported tool compared to conventional estimation methods?**).

In order to address the accuracy, we asked the development teams to perform the actual implementation. The accuracy was then calculated as the ratio between the estimate of task or user story and the actual effort, reported by the developers (see Equation (3)). As stated in Section 5, the developers tracked their development time based on tasks. If a user story was accepted, then the user story’s actual effort was calculated as a sum of the fulfilled task efforts (see Equation (2)).

$$\text{Estimation Accuracy(US}_i\text{)} = \frac{\text{Actual effort(US}_i\text{)}}{\text{Estimate(US}_i\text{)}} \times 100\% \quad (3)$$

After the implementation, 32 user stories were accepted, performed by implementing 101 tasks, which were our empirical data body to carry out the accuracy analysis. The accuracy of the experimental group estimates was really low: 16% on average (standard deviation of 9.2%). This is not an acceptable level of accuracy, as expected by the industry [32]. To put this into perspective, the control group’s effort accuracy was 60% (standard deviation 24.5%), which correlates to the expected accuracy in the industry [32]. With this, we answered research question 2 partially (RQ 2: How accurate is the effort estimation generated by an AI-supported tool compared to conventional estimation methods?). The accuracy per user stories and groups is shown in Figure 9.

Since the estimation accuracy calculation is carried out with two variables (estimated effort, actual effort), a legitimate debate can also question the correctness of the reported actual effort. To minimise the risk of misreporting, as already explained in Section 4, we hid the estimates from the development teams before the implementation phase. This approach is not uncommon in agile development, where estimates are primarily used during planning meetings to help teams make fair and informed commitments. During the development phase, these estimates are typically “hidden” to ensure developers can focus on delivering high-quality solutions without being influenced by prior estimations, thereby maintaining the integrity of the development process.

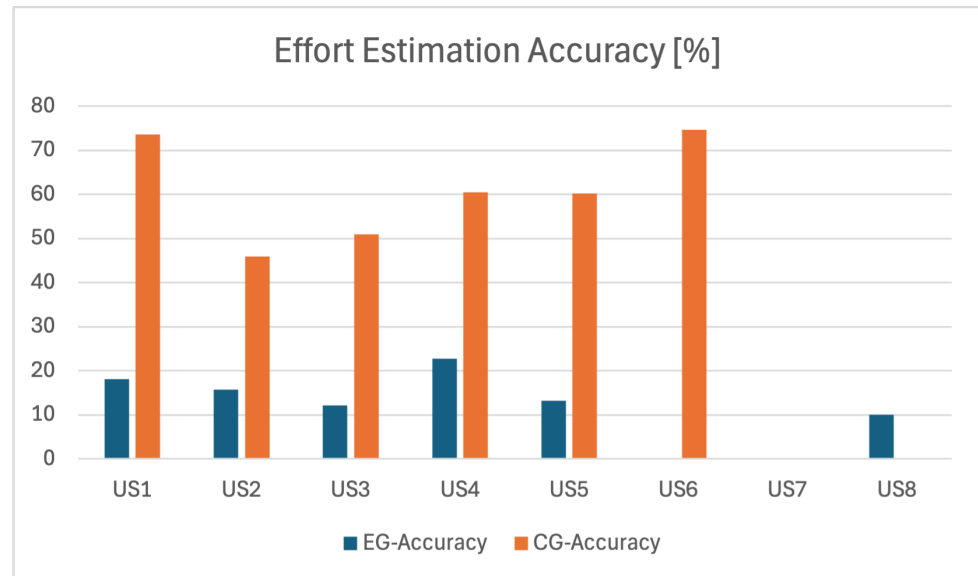


Figure 9. Estimation accuracy in the experimental (EG) and control (CG) groups per user story, in percent.

Another interesting insight, supporting the correctness of reported hours, can be seen in Table 2 of Section 5: regardless of the different task lists and estimates, the development teams of both groups demonstrated similar efforts for implementing user stories to an acceptable level (2 and 1.92 h). All the estimates and actual efforts of both groups per user story are shown in Figure 10, where we can see clearly the similarities between the control group estimates, control group actual effort and experimental group actual effort. The experimental group estimates are obvious outliers.

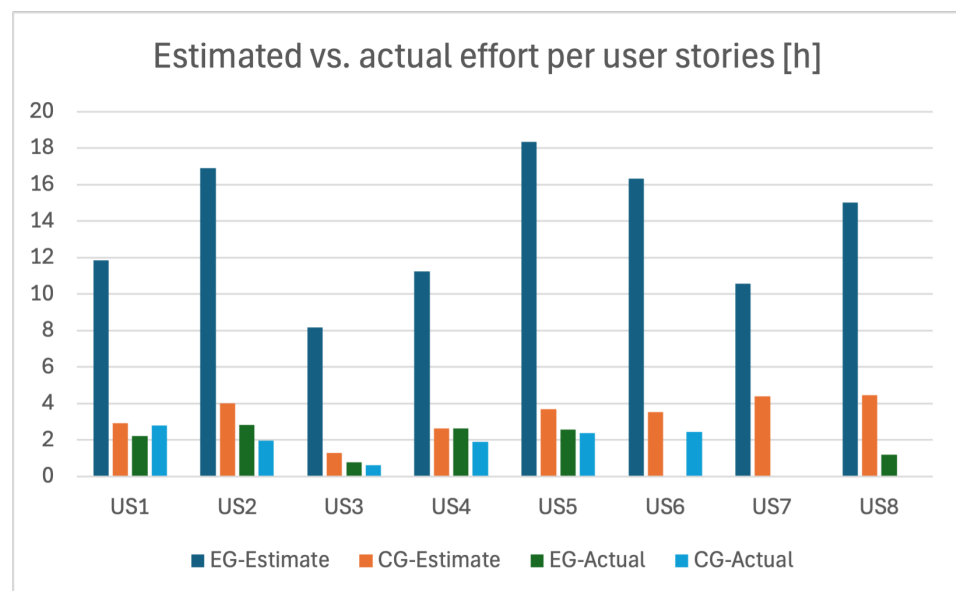


Figure 10. Estimated and actual effort in the experimental (EG) and control (CG) groups per user story in hours.

When exposing development-related empirical data to statistical tests, they showed us interesting insights. As already stated, the estimates from the experimental and control groups demonstrated significant differences. When we compared the estimated and actual effort of the experimental group further, a significant difference was clearly proven: the two-tailed student *t*-test resulted in a *p*-value of 2.2×10^{-9} , *t*-value -7.67 . This means

that the estimates in the experimental group were all but accurate. When verifying if the development effort was comparable in both groups, the statistical test suggests similarity (a significant difference cannot be proven— $p = 0.675$; $t = 0.424$). The t -test also suggested similarity between the estimated and actual effort in the control group: $p = 0.111$; $t = -1.646$.

Table 3. Statistical tests (Student t -test) summary.

Data Sets	p -Value	t -Value	Interpretation
US estimates—CG US estimates—EG	3.03×10^{-7}	6.676	Estimates are proven to have significant differences
US actual effort—EG US estimates—EG	2.2×10^{-9}	7.67	Estimates are significantly different compared to actual effort
US actual effort—CG US estimates—CG	0.111	−1.646	Similarity is suggested between the estimates and actual effort
US actual effort—CG US actual effort—EG	0.675	0.424	Similarity is suggested between the actual efforts in both groups

Let us summarise the discussion by answering research question 3 (**RQ3: Which aspects of effort estimation can be backed by an AI-supported tool?**). Based on RQ1, task generation is a candidate activity that can be improved by generative AI tool support. However, our experiment empirical data did not demonstrate the possibility of carrying it out automatically but, rather, as a helper tool. This outcome is also aligned with the post-test questionnaire results: the participants unanimously saw this activity to be carried out with the combination of conventional methods and AI-supported tools.

When discussing effort estimation, our experimentation shows clearly that the generative AI tools that we used were not mature enough to be used, since the accuracy was not only low but also, importantly, lower than the methods and tools that are used today in the software development industry. This insight has also been reported clearly by the participants in the post-test questionnaire: the majority of the participants were in doubt about the generative-AI-supported estimates or were (more than half) against AI-supported effort estimation (see Section 5 for details).

7. Threats of Validity

7.1. Construct Validity

The research fields of AI, especially generative AI and large language models, are in ascent at the moment. Our experiment, carried out with the tools based on the current state of the art in the field, might not be relevant in years to come or even when improved methods and tools are released. In addition, the tool that we used in the experiment was one of the best on the market. However, the tool was used in the form for a general audience. We believe that if such a tool would be fine-tuned to a particular company and/or domain, it would have performed better. It would also perform better when data feedback in terms of accuracy would be used in future assessments. Unfortunately, the tool does not allow for such adjustments.

In addition, developers' pre-knowledge and experience was implicitly included in the effort estimation due to self-assessment in control group. However, this was not the case in the experimental group, using generative-AI-based estimation, as the prompts do not explicitly contain all the information about the developers' experience. Having generative-AI-based tools aware of pre-knowledge would improve comparability between the groups. When the tools support this functionality, we plan to repeat our research.

7.2. Internal Validity

Although the participants in the research were 4th semester software engineering students who we train from day one in the technologies that were used during the experiment, the fact that they were not full-time developers might affect the results negatively.

Our experimental setup carries some limitations. While the planning meeting closely simulated real industry settings, the development phase was brief, and user stories and tasks were smaller in scale than in typical projects. This was intentional to maintain a controlled environment but may limit generalizability. Not all user stories were implemented, reflecting the agile practice of prioritizing work based on capacity. The teams selected stories in line with iteration commitment, a core agile principle. While this setup may not fully replicate larger projects, it captures key aspects of agile practices, though the limited empirical data might reduce the significance of the results.

7.3. External Validity

We limited ourselves to GitLab Duo for the controlled experiment, which may pose a threat when generalizing our findings. GitLab Duo was identified in 2024, along with Atlassian Intelligence, ClickUp and OpenText, as a leading tool for iteration planning using generative AI. Based on this and our own testing, we chose GitLab Duo, integrated with GitLab Enterprise, which gave us confidence in the tool's relevance. However, using a single tool may limit the broader applicability of our results.

8. Conclusions

An effort estimation, especially in agile software development, is a cornerstone activity. It enables fair judgement of the effort needed to perform a certain development activity and, importantly, to give sincere commitments on the iteration scope. Since proper estimates minimise schedule-, finance-, and resource-related risks, they are carried out carefully, using proven methods and tools. However, effort estimation can not only be time consuming but also occupy the most brilliant minds from the development teams. This is why, for the recent decades, several approaches emerged to introduce help or even automation to the realm of effort estimation. The recent breakthrough in AI, especially in generative AI, that is reshaping a complete software development discipline, implies that novel tools can also be used to estimate effort more efficiently. To put this tempting idea into trial, we designed and executed a controlled experiment.

Considering the key threats that our research is facing (approaching low maturity, estimating with limited pre-knowledge information, student participants, short iteration), we can outline several significant insights. The research takeaways show clearly that the currently leading helper tools, integrated in software development platforms (we used GitLab Duo), are not mature enough to be a replacement for a human estimator. The automated effort estimates, averaged at 16% accuracy, are far from usable in an industry setting. However, our empirical data showed that they can, even in their present form, be used fruitfully as a helper while splitting the requirements of development tasks—let it be task generation or as a help to see if important tasks were left out during the iteration planning.

While analysing the empirical data, we found three clusters of estimates that contributed importantly to the low accuracy. The first tasks were the ones that were not required explicitly and were usually left out by human assessors. Those tasks can be included by conventional task generation activity and re-estimated by human assessors. The second group consisted of tasks that were included in the automatically generated task lists due to ignoring the context of the effort estimation. The last group of tasks that contributed to low accuracy were the ones that were needed explicitly but were overestimated, obviously planned in terms of a worst-case scenario or ignoring the developers' competences.

The research in the field of automating the effort estimation activity is far from finished. Our research into generative AI potential in the field will be extended and possibly repeated after the tools become more mature, to see their impact. The future work we plan to do

is to define a sound set of activities built around generative-AI-enabled tools in order to improve effort estimation. The research method to validate them might not only be based on controlled experiments. We plan to test our ideas on industry projects, where development teams exercise effort estimation with discipline and consistency. In this setting, we might experiment with a known codebase at a certain point of time, known requirements that were implemented, and the known effort that was used to implement them. This is how we would have invaluable empirical data used to test competing methods of automated effort estimation. However, we are aware of the threats that this and future research are facing when generalizing the findings.

Currently, we can conclude confidently that generative-AI-based tools are not at the level where they could be used efficiently to estimate effort in software engineering. However, with vigilance, they can be a valuable companion while splitting requirements into development tasks. In conclusion, while our research does not advocate for the use of LLM tools in critical tasks like iteration planning, we recognize their growing presence in the industry. Our evaluation shows that the tools are not yet mature enough to replace human estimators. Although these tools are readily available and often misused for such purposes, our findings emphasize the need for caution and further development before the tools can be relied upon in complex planning tasks.

Author Contributions: Conceptualization, L.P., V.S. and T.B.; methodology, L.P., V.S. and T.B.; software, L.P.; validation, L.P., V.S. and T.B.; formal analysis, L.P. and V.S.; resources, L.P., V.S. and T.B.; data curation, L.P. and V.S.; writing—original draft preparation, L.P., V.S. and T.B.; writing—review and editing, L.P., V.S. and T.B.; visualization, L.P.; supervision, L.P.; project administration, L.P. and T.B.; funding acquisition, L.P. All authors have read and agreed to the published version of the manuscript.

Funding: The authors acknowledge the financial support from the Slovenian Research and Innovation Agency (Research Core Funding No. P2-0057).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Cambridge University Press. Cambridge Dictionary. Available online: <https://dictionary.cambridge.org> (accessed on 9 August 2024).
2. PMI. (Ed.) *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 5th ed.; Project Management Institute: Newtown Square, PA, USA, 2013.
3. Brooks, F., Jr. *The Mythical Man-Month: Essays on Software Engineering*; Addison-Wesley: Boston, MA, USA, 1995.
4. Fernández-Diego, M.; Méndez, E.R.; González-Ladrón-De-Guevara, F.; Abrahão, S.; Insfran, E. An Update on Effort Estimation in Agile Software Development: A Systematic Literature Review. *IEEE Access* **2020**, *8*, 166768–166800. [\[CrossRef\]](#)
5. Usman, M.; Mendes, E.; Neiva, F.; Britto, R. Effort Estimation in Agile Software Development: A Systematic Literature Review. In Proceedings of the 10th International Conference on Predictive Models in Software Engineering, Turin, Italy, 17 September 2014. [\[CrossRef\]](#)
6. Peluffo-Ordóñez, D.; Sierra Martínez, L.; Timana, J.; Piñeros, C. Effort Estimation in Agile Software Development: A Systematic Map Study. *Inge CUC* **2023**, *14*, 22–36. [\[CrossRef\]](#)
7. Schweighofer, T.; Kline, A.; Pavlič, L.; Heričko, M. How is Effort Estimated in Agile Software Development Projects? In Proceedings of the SQAMIA, 5th Workshop of Software Quality, Analysis, Monitoring, Improvement, and Applications, Budapest, Hungary, 29–31 August 2016.
8. Cohn, M. *Agile Estimating and Planning*; Addison-Wesley Professional: Boston, MA, USA, 2005.
9. Dinwiddie, G. *Software Estimation Without Guessing*; Pragmatic Bookshelf: Raleigh, NC, USA, 2019.
10. Munialo, S.; Muketha, G. A Review of Agile Software Effort Estimation Methods. *Int. J. Comput. Appl. Technol. Res.* **2016**, *5*, 612–618. [\[CrossRef\]](#)
11. Digital.AI. 15th State of Agile Report. Available online: <https://digital.ai/resource-center/analyst-reports/state-of-agile-report> (accessed on 9 August 2024).

12. Schwaber, K.; Sutherland, J. The Definitive Guide to Scrum: The Rules of the Game. Retrospectiva del Sprint de Nexus. 2017. Available online: <https://scrum.org> (accessed on 3 November 2024).
13. Agile Alliance. What is Scrumban? Available online: <https://www.agilealliance.org/scrumban/> (accessed on 3 November 2024).
14. Large Scale Scrum (LeSS). Available online: <https://less.works/> (accessed on 3 November 2024).
15. Scaled Agile Framework. Available online: <https://scaledagileframework.com/> (accessed on 3 November 2024).
16. Extreme Programming (XP). Available online: <http://www.extremeprogramming.org/> (accessed on 3 November 2024).
17. Feature-Driven Development (FDD). Available online: <https://agilemodeling.com/essays/fdd.htm> (accessed on 3 November 2024).
18. Alsaadi, B.; Saeedi, K. Data-driven effort estimation techniques of agile user stories: A systematic literature review. *Artif. Intell. Rev.* **2022**, *55*, 5485–5516. [\[CrossRef\]](#)
19. Hannigan, T.R.; McCarthy, I.P.; Spicer, A. Beware of botshit: How to manage the epistemic risks of generative chatbots. *Bus. Horizons* **2024**, *67*, 471–486. [\[CrossRef\]](#)
20. Bhat, M.; Haight, C.; Blosen, B. *How Platform Engineering Teams Can Augment DevOps with AI*; Gartner Research: Stamford, CT, USA, 2024.
21. Finnie, G.; Wittig, G. AI tools for software development effort estimation. In Proceedings of the 1996 International Conference Software Engineering: Education and Practice, Dunedin, New Zealand, 24–27 January 1996; pp. 346–353. [\[CrossRef\]](#)
22. Jadhav, A.; Shandilya, S.K. Reliable machine learning models for estimating effective software development efforts: A comparative analysis. *J. Eng. Res.* **2023**, *11*, 362–376. [\[CrossRef\]](#)
23. Tran, T.N.; Tran, H.T.; Nguyen, Q.N. Leveraging AI for Enhanced Software Effort Estimation: A Comprehensive Study and Framework Proposal. In Proceedings of the 2023 International Conference on the Cognitive Computing and Complex Data (ICCD), Huaian, China, 21–22 October 2023; pp. 284–289. [\[CrossRef\]](#)
24. Koskinen, K. AI-Assisted Software Development Effort Estimation. Master’s Thesis, Department of Computing, University of Turku, Turku, Finland, 2021.
25. Hartenstein, S.; Leroy Johnson, S.; Schmietendorf, A. *Towards a Fast Cost Estimation Supported by Large Language Models*; Technical Report; COSMIC Sizing: Chambly, QC, Canada, 2024.
26. Coelho, C.N., Jr.; Xiong, H.; Karayil, T.; Koratala, S.; Shang, R.; Bollinger, J.; Shabar, M.; Nair, S. Effort and Size Estimation in Software Projects with Large Language Model-based Intelligent Interfaces. *arXiv* **2024**, arXiv:2402.07158. <http://arxiv.org/abs/2402.07158>.
27. Arman, A.; Di Reto, E.; Mecella, M.; Santucci, G. An Approach for Software Development Effort Estimation Using ChatGPT. In Proceedings of the 2023 IEEE International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE), Paris, France, 14–16 December 2023; pp. 1–7. [\[CrossRef\]](#)
28. Ebrahim, E.; Sayed, M.; Youssef, M.; Essam, H.; El-Fattah, S.A.; Ashraf, D.; Magdy, O.; ElAdawi, R. AI Decision Assistant ChatBot for Software Release Planning and Optimized Resource Allocation. In Proceedings of the 2023 IEEE International Conference on Artificial Intelligence Testing (AITest), Athens, Greece, 17–20 July 2023; pp. 55–60.
29. Barcaui, A.; Monat, A. Who is better in project planning? Generative artificial intelligence or project managers? *Proj. Leadersh. Soc.* **2023**, *4*, 100101. [\[CrossRef\]](#)
30. Wohlin, P.; Runeson, M.H.M.C.O.B.R.; Wesslén, A. *Experimentation in Software Engineering*; Springer: Berlin/Heidelberg, Germany, 2012.
31. Ko, A.J.; LaToza, T.D.; Burnett, M.M. A practical guide to controlled experiments of software engineering tools with human participants. *Empir. Softw. Eng.* **2015**, *20*, 110–141. [\[CrossRef\]](#)
32. Jørgensen, M.; Teigen, K.H.; Moløkken, K. Better sure than safe? Over-confidence in judgement based software development effort prediction intervals. *J. Syst. Softw.* **2004**, *70*, 79–93. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.